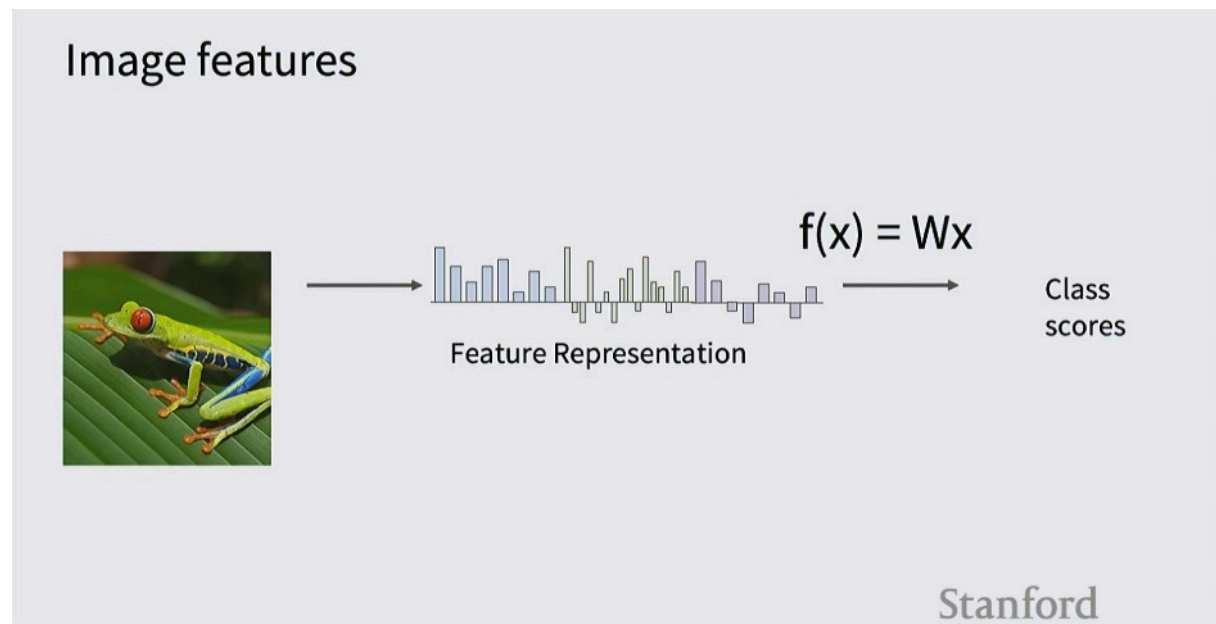


Lecture 5 : Image Classification with CNNs

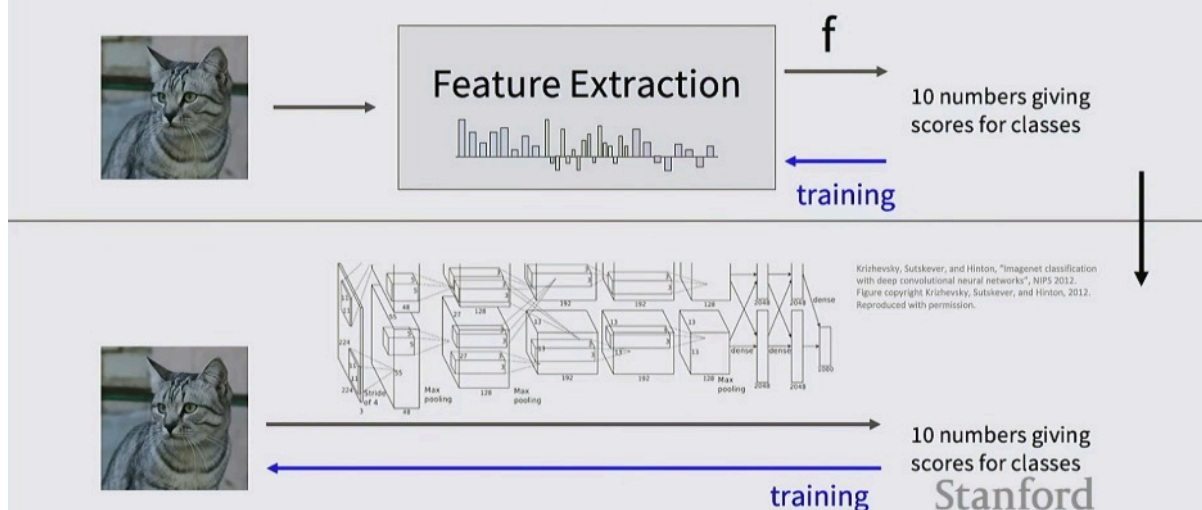
CNN은 기존 신경망(완전 연결 층 + 활성화 함수)에 **합성곱 층(Convolution Layer)**과 **풀링 층(Pooling Layer)**을 추가한 구조이다. 이미지를 1차원으로 펼치지 않고 **2차원 공간 구조를 유지한 채** 처리한다는 점이 핵심이다.

특징 추출 방식과 End-to-End 학습



- 딥러닝 이전에는 사람이 설계한 **특징 추출 함수**로 이미지를 특징 벡터로 변환한 뒤, 이를 선형 분류기에 입력하였다.

Image features vs. ConvNets

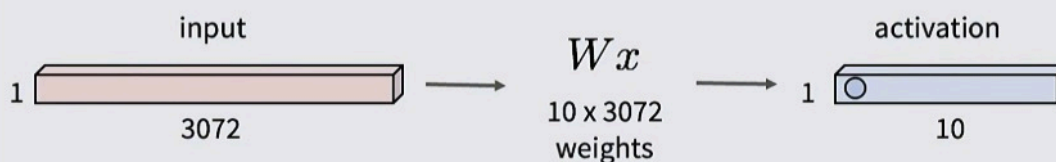


방식	사람이 설계하는 부분	학습되는 부분
특징 추출 + 분류기	특징 추출기 전체	마지막 분류기
CNN (End-to-End)	신경망 구조	픽셀부터 최종 점수까지 전부

Convolution Layer

Recap: Fully Connected Layer

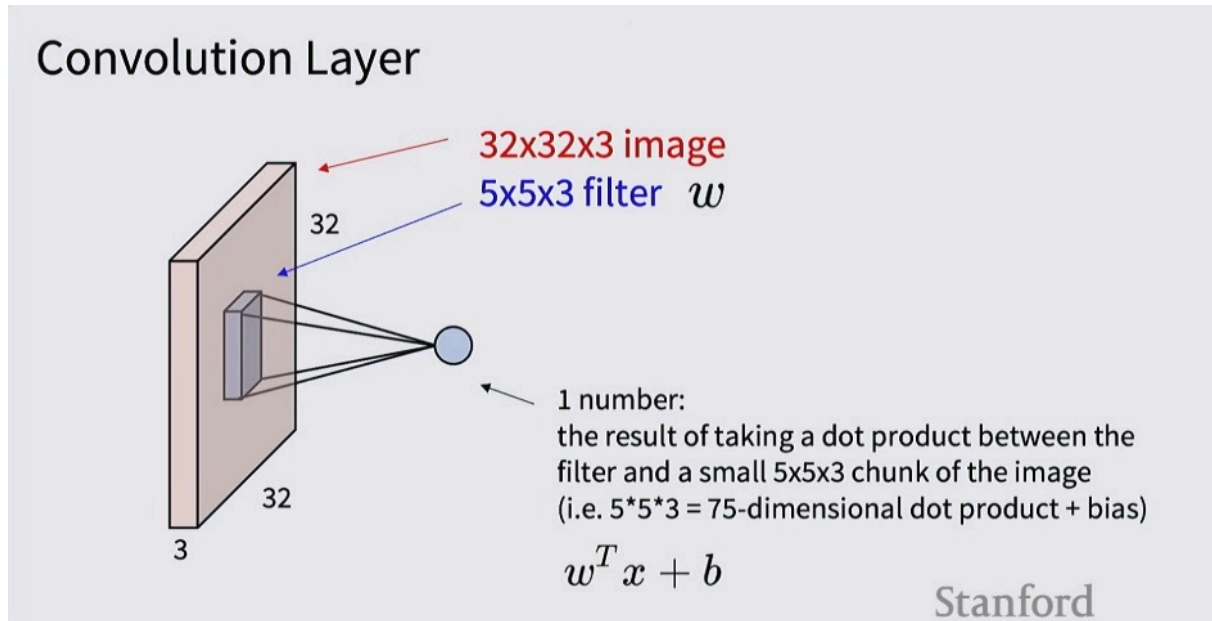
32x32x3 image -> stretch to 3072 x 1



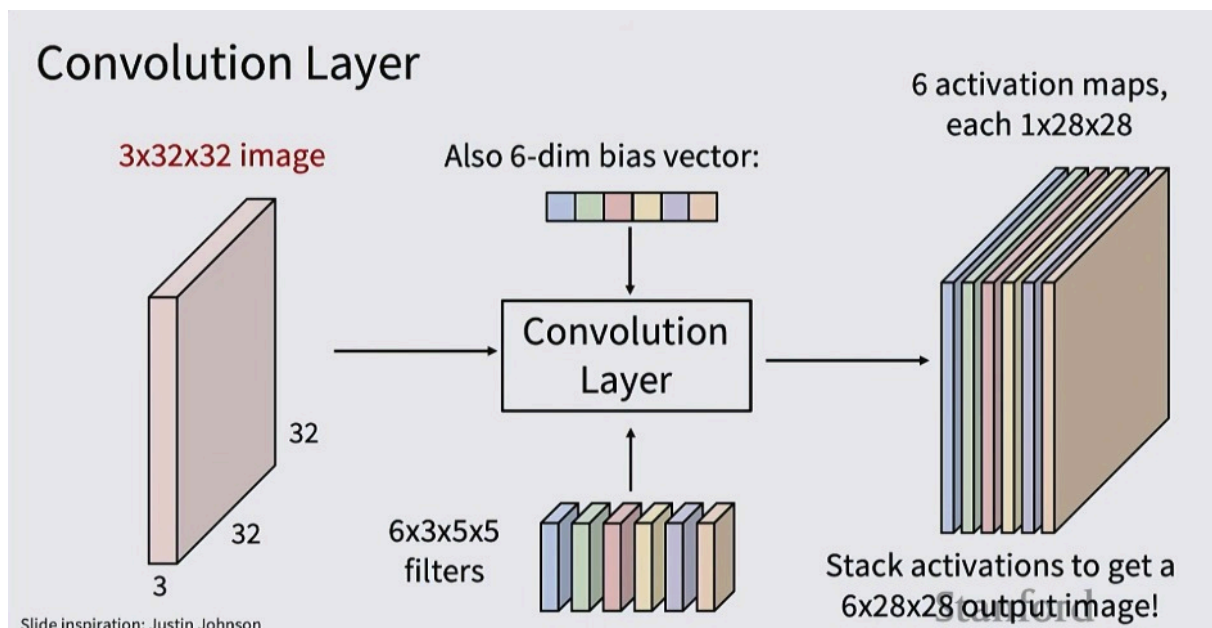
Stanford

- 완전 연결 층은 $32 \times 32 \times 3$ 이미지를 3072 차원 벡터로 펼친 뒤 W 와 곱한다.

- 출력의 각 원소 = 입력 전체와 템플릿의 내적
- (-) : 이미지를 펼치는 순간 **2D 공간 구조가 사라진다**
- 템플릿 매칭의 아이디어는 유지하되, **작은 템플릿을 이미지 위에서 이동**



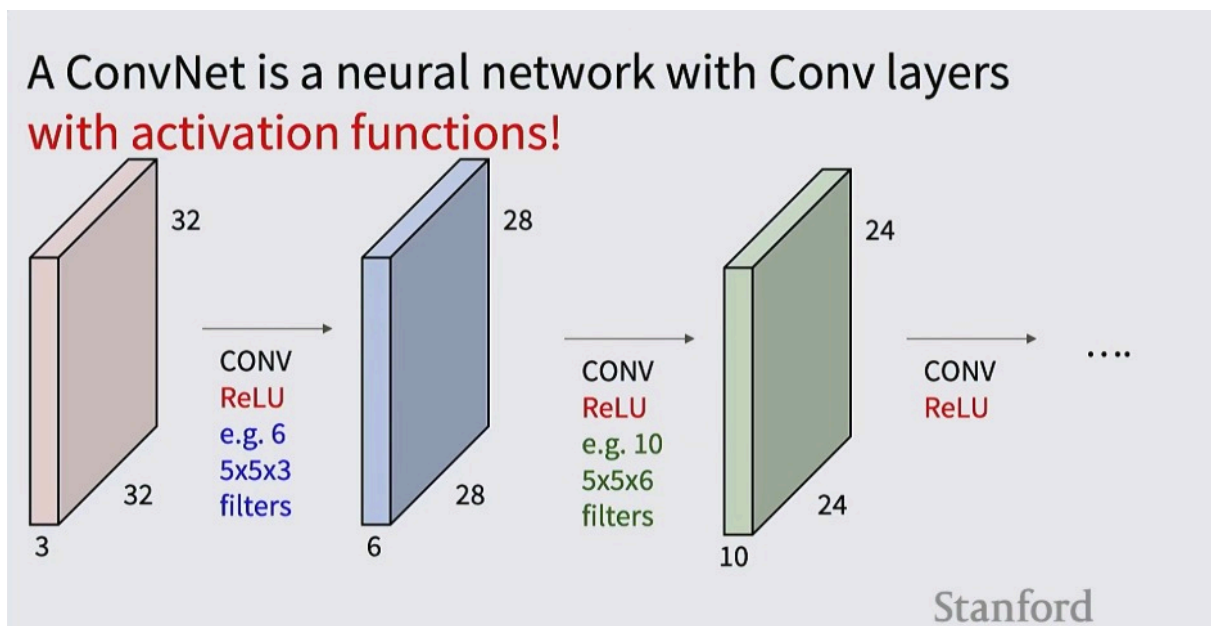
- 입력 $32 \times 32 \times 3$, 필터 $5 \times 5 \times 3$. 필터의 깊이는 항상 **입력의 채널 수와 같다**.
- 필터를 이미지의 한 위치에 겹쳐 $w^T x + b$ 를 계산하면 숫자 하나가 나온다.
- 필터를 모든 위치로 이동하며 계산한 결과 = **활성화 맵**



- 필터 6개를 사용하면 활성화 맵 6장이 쌓여 출력은 $6 \times 28 \times 28$ 이 된다.

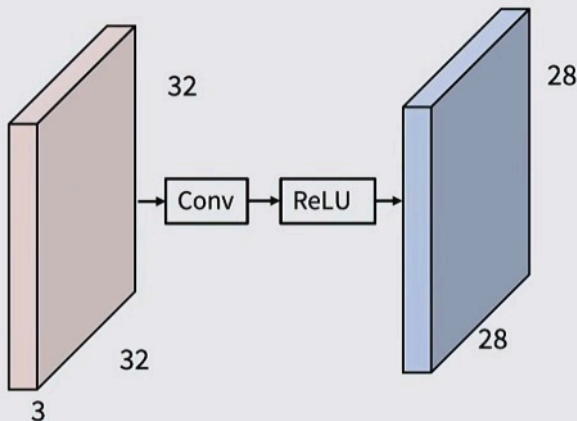
- (출력 채널 수 = 필터 개수)
- 편향은 필터마다 하나씩 존재.
- 필터 값은 사람이 정하지 않고 역전파와 경사하강법으로 학습된다.
- 필터 개수와 크기는 하이퍼파라미터이다.

합성곱 층 쌓기와 학습된 필터

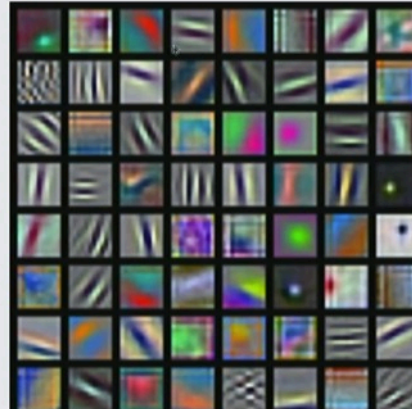


- 다음 층의 필터 깊이는 이전 층의 출력 채널 수와 같아야 한다.
 - ex. $6 \times 28 \times 28 \rightarrow 5 \times 5 \times 6$ 필터 10개 $\rightarrow 10 \times 24 \times 24$
- 합성곱은 선형 연산이므로 그대로 쌓으면 하나의 합성곱과 같아져서 layer 사이에 **ReLU 등의 활성화 함수**를 넣는다.

What do Conv filters learn?



First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)



AlexNet: 64 filters, each 3x11x11

:AlexNet 첫 번째 층의 필터 64개($11 \times 11 \times 3$)

- 첫 층은 **색 대비와 방향별 edge**를 학습, Hubel & Wiesel 실험의 결과와 유사.
- 깊은 층으로 갈수록 눈, 바퀴, 글자 등 **더 크고 복잡한 구조**에 반응.
- 필터를 무작위로 초기화해야 대칭이 깨져서 각 필터가 서로 다른 특징을 학습.

출력 크기

Convolution: Spatial Dimensions

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7
Filter: 3x3
Output: 5x5

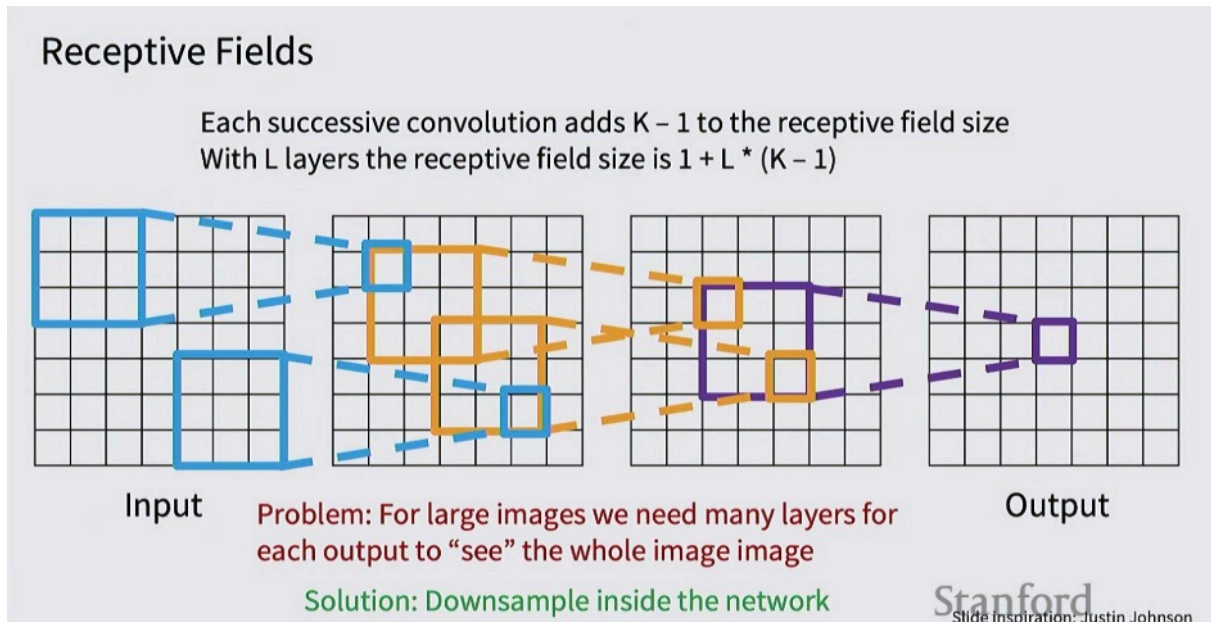
Problem: Feature maps shrink with each layer!

In general
Input: W
Filter: K
Padding: P

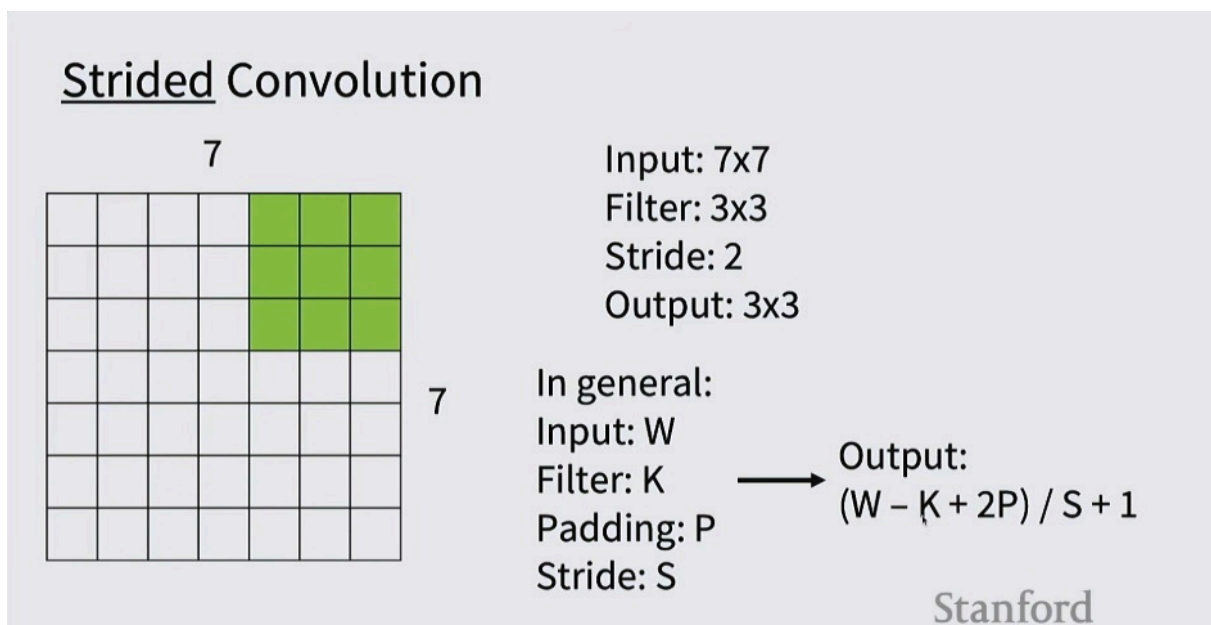
Output: $W - K + 1 + 2P$

Solution: Add **padding** around the input before sliding the filter

- 입력 크기 W , 필터 크기 K 일 때 출력은 $W - K + 1$. 레이어를 지날수록 특징 맵이 줄어든다.
- **패딩** : 입력 테두리에 0을 덧붙인다. 출력은 $W - K + 1 + 2P$, 보통 $P = (K - 1) / 2$ 로 설정해 입력과 출력의 크기를 같게 유지.

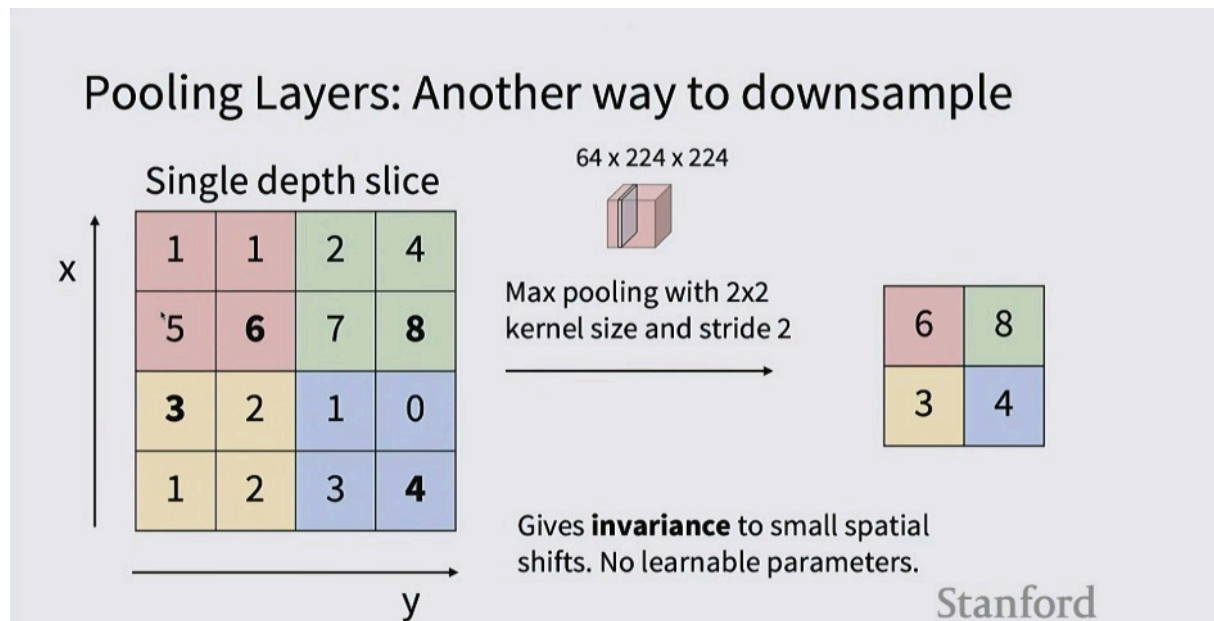


- **수용 영역** : 출력의 한 점이 원본 입력에서 참조하는 영역의 크기
 - $K \times K$ 합성곱을 L 층 쌓으면 수용 영역은 $1 + L(K - 1)$.
 - 층 수에 선형으로만 증가하므로, 큰 이미지 전체를 보려면 **다운샘플링**이 필요



- **스트라이드**: 필터를 이동하는 간격. stride 2는 특징 맵을 절반으로 줄여 수용 영역을 빠르게 증가시킨다.

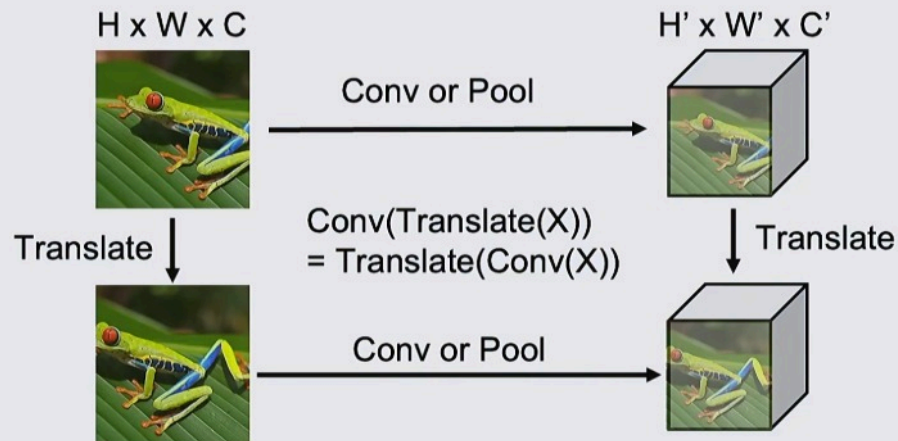
Pooling Layer



- 연산 비용이 낮고 **학습 파라미터가 없는** 다운샘플링 방법.
- 채널마다 독립적 적용 → 채널 수는 유지.
- **최대 풀링** : 2×2 , stride 2로 겹치지 않는 영역마다 최댓값만 남긴다.
- 출력 크기 : $(H - K) / S + 1$. 가장 일반적인 설정은 $K = 2, S = 2$
- 최대 풀링은 그 자체로 비선형 연산이다.
- 일반적인 구성 : [합성곱 → ReLU] $\times n$ → 풀링을 반복 후 마지막에 완전 연결 층으로 분류.

Translation Equivariance(평행 이동 등변성)

Convolution and Pooling: Translation Equivariance



Intuition: Features of images don't depend on their location in the image

- 이미지를 이동 후 합성곱한 결과 = 합성곱 후 이동한 결과
- $\text{Conv}(\text{Translate}(X)) = \text{Translate}(\text{Conv}(X))$
- = 추출되는 특징은 **내용에만 의존. 위치 의존 X**. 같은 필터를 이미지 전체에 적용하기 때문에 생기는 성질.
- 기존 방식이 이미지에 대한 사전 지식을 특징 추출 함수에 담았다면, CNN은 이를 **층의 구조 자체**에 담고 필터 값은 데이터로 학습한다.